



04-DARS · 14 TADAN

JavaScript asoslari

O'zgaruvchilar, ma'lumot turlari, operatorlar, shartlar, sikllar, funksiyalar, scope va closure, massiv va obyekt metodlari, sinflar, modullar va xatolar bilan ishlash.

Boshlang'ich–O'rta · ~150 daqiqa

JavaScript — webning yagona dasturlash tili: u brauzerda ishlaydi va sahifani "jonlantiradi". Node.js tufayli esa serverda ham ishlaydi — ya'ni full-stack dasturchi **bitta til** bilan frontend va backendni yoza oladi. Bu dars eng uzun va eng muhim darslardan biri: keyingi barcha mavzular shu yerdagi tushunchalarga tayanadi.

JavaScript'ni ishga tushirish

- **Brauzer konsoli:** `F12` → Console. Tez tajribalar uchun ideal.
- **HTML ichida:** `<script src="app.js" defer></script>`
- **Node.js:** `node app.js` (terminalda)

JavaScript

```
console.log('Salom, dunyo!');           // oddiy chiqarish
console.table([ { ism: 'Ali', yosh: 20 }, { ism: 'Vali', yosh: 22 } ]); // jadval
ko'rinishida
console.error('Xato xabari');           // qizil rangda
```

O'zgaruvchilar: let, const, var

JavaScript

```
const PI = 3.14159;      // qayta tayinlab bo'lmaydi
let yosh = 25;           // qayta tayinlash mumkin
yosh = 26;               // ✓
// PI = 3;               // ✗ TypeError: Assignment to constant variable

const user = { ism: 'Aziz' };
user.izm = 'Anvar';      // ✓ const – havolani qotiradi, obyekt ichini emas!
// user = {};           // ✗
```

	var	let	const
Ko'rinish doirasi	Funksiya	Blok { }	Blok { }
Qayta tayinlash	Ha	Ha	Yo'q
Hoisting	Ha, undefined bilan	Ha, lekin TDZ (e'londan oldin ishlatib bo'lmaydi)	Ha, TDZ

 **Qoida**

Doim `const` dan boshlang. Qiymat o'zgarishi kerak bo'lsagina `let` ga almashtiring. `var` ni umuman ishlatmang — u eski kodlarda uchraydi va kutilmagan xatolarga sabab bo'ladi.

Nomlash: `camelCase` (`userName`, `totalPrice`), konstantalar uchun ba'zan `UPPER_CASE`, sinflar uchun `PascalCase`. Nom ma'noli bo'lsin: `x` emas, `kunlikDaromad`.

Ma'lumot turlari

JavaScript'da 8 ta tur bor: 7 ta **primitiv** va **obyekt**.

Tur	Misol	typeof
-----	-------	--------

String (satr)	'salom', "dunyo", `shablon \${x}`	"string"
Number (son)	42, 3.14, -7, NaN, Infinity	"number"
BigInt	9007199254740993n	"bigint"
Boolean	true, false	"boolean"
undefined	qiymat berilmagan o'zgaruvchi	"undefined"
null	"ataylab bo'sh" qiymat	"object" (tarixiy xato!)
Symbol	Symbol('id') — noyob kalit	"symbol"
Object	{}, [], funksiyalar, Date, Map ...	"object" / "function"

Primitiv va havola (reference) turlar

Bu farqni tushunish ko'plab xatolarning oldini oladi:

JavaScript

```
// Primitivlar QIYMAT bo'yicha nusxalanadi
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 — o'zgarmadi

// Obyektlar HAVOLA bo'yicha nusxalanadi
const user1 = { ism: 'Ali' };
const user2 = user1; // ikkalasi BITTA obyektga ishora qiladi
user2.ism = 'Vali';
console.log(user1.ism); // 'Vali' — o'zgardi!

// Haqiqiy nusxa olish
const copy = { ...user1 }; // sayoz (shallow) nusxa
const deep = structuredClone(user1); // chuqur (deep) nusxa

// Taqqoslash ham havola bo'yicha
console.log({ a: 1 } === { a: 1 }); // false — ikki xil obyekt
```

Satrlar bilan ishlash

JavaScript

```
const ism = 'Malika';
const salom = `Salom, ${ism}! Bugun ${new Date().getFullYear()}-yil.`; //
template literal

const s = '  JavaScript Dasturlash  ';
s.length; // 25
s.trim(); // 'JavaScript Dasturlash'
s.toUpperCase(); // ' JAVASCRIPT DASTURLASH '
s.includes('Script'); // true
s.trim().startsWith('Java'); // true
s.indexOf('a'); // 3
s.trim().slice(0, 10); // 'JavaScript'
'a,b,c'.split(','); // ['a', 'b', 'c']
'ha'.repeat(3); // 'hahaha'
'5'.padStart(3, '0'); // '005'
'salom dunyo'.replaceAll('o', '0'); // 'sa10m duny0'
```

Sonlar va Math

JavaScript

```
0.1 + 0.2; // 0.30000000000000004 (!) – kasr sonlar ikkilik
tizimda aniq emas
(0.1 + 0.2).toFixed(2); // '0.30'
Number('42'); // 42
Number('42px'); // NaN
parseInt('42px'); // 42
parseFloat('3.5kg'); // 3.5
Number.isNaN(NaN); // true
Math.round(4.5); // 5
Math.floor(4.9); // 4
Math.ceil(4.1); // 5
Math.max(3, 7, 2); // 7
Math.floor(Math.random() * 10) + 1; // 1..10 oralig'ida tasodifiy butun son
(1234567.891).toLocaleString('uz-UZ'); // '1 234 567,891'
```



Pul bilan ishlaganda kasr sonlardan saqlaning: summani **tiyinlarda (butun son)** saqlang (`1999 so'm 50 tiyin → 199950`) yoki serverda aniq o'nlik turlaridan (SQL'dagi `NUMERIC`) foydalaning.

Turlarni o'zgartirish (type coercion)

JavaScript

```
'5' + 3;      // '53' - + operatori satr bo'lsa birlashtiradi
'5' - 3;      // 2    - boshqa operatorlar songa aylantiradi
'5' * '2';    // 10
true + 1;     // 2

// Falsy qiymatlar (shartda false deb hisoblanadi) – atigi 8 ta:
// false, 0, -0, 0n, '', null, undefined, NaN
// Qolgan HAMMASI truthy: '0', 'false', [], {}, ' '

// = va ===
0 = '';      // true 🤖 (tur o'zgartiriladi)
0 === '';   // false ✅ (tur ham tekshiriladi)
null = undefined; // true
null === undefined; // false
```



Har doim `===` va `!==` dan foydalaning. `=` ning tur o'zgartirish qoidalari chalkash va xatolarga olib keladi.

Operatorlar

JavaScript

```
// Arifmetik: + - * / % (qoldiq) ** (daraja)
10 % 3;      // 1
2 ** 10;     // 1024

// Qisqa tayinlash
let x = 5;
x += 2; x *= 3; x++; // 22

// Mantiqiy
true && false; // false (VA)
true || false; // true  (YOKI)
!true;         // false (EMAS)

// Qisqa tutashuv (short-circuit) – real kodda juda ko'p ishlatiladi
const ism = kiritilganIsm || 'Mehmon'; // chap tomon falsy bo'lsa – o'ng
const soni = kiritilganSon ?? 10;      // FAQAT null/undefined bo'lsa –
o'ng (0 ni saqlaydi!)
isLoggedIn && showDashboard();         // chap true bo'lsa bajaradi

// Ixtiyoriy zanjir (optional chaining)
const shahar = user?.address?.city;    // address yo'q bo'lsa xato emas,
undefined
user.sayHi?.();                        // metod bo'lsa chaqiradi

// Uchlik (ternary) operator
const status = yosh ≥ 18 ? 'katta' : 'voyaga yetmagan';
```

Shartlar va sikllar

JavaScript

```
// if / else if / else
const ball = 78;
if (ball ≥ 86) {
  console.log("A'lo");
} else if (ball ≥ 71) {
  console.log('Yaxshi');
} else if (ball ≥ 56) {
  console.log('Qoniqarli');
} else {
  console.log('Qoniqarsiz');
}

// switch – bitta qiymatni ko'p variantga solishtirish
switch (kun) {
  case 'shanba':
  case 'yakshanba':
    console.log('Dam olish kuni');
    break; // break unutilsa, keyingi case ham bajariladi!
  default:
    console.log('Ish kuni');
}

// Sikllar
for (let i = 1; i ≤ 5; i++) console.log(i);

const mevalar = ['olma', 'nok', 'uzum'];
for (const meva of mevalar) console.log(meva); // massiv QIYMATLARI

const narxlar = { olma: 12000, nok: 15000 };
for (const kalit in narxlar) console.log(kalit, narxlar[kalit]); // obyekt KALITLARI

let n = 3;
while (n > 0) { console.log(n); n--; }

// break – siklni to'xtatish, continue – keyingi iteratsiyaga o'tish
for (let i = 1; i ≤ 10; i++) {
  if (i % 2 === 0) continue;
  if (i > 7) break;
}
```

```
console.log(i); // 1, 3, 5, 7  
}
```

Funksiyalar

Funksiya — nomlangan, qayta ishlatiladigan kod bloki. JavaScript'da funksiyalar **birinchi darajali qiymatlar** (first-class): ularni o'zgaruvchiga saqlash, argument sifatida berish va funksiyadan qaytarish mumkin.

```
// 1. Function declaration – hoisting tufayli e'londan oldin ham chaqirish
mumkin
function qoshish(a, b) {
  return a + b;
}

// 2. Function expression
const ayirish = function (a, b) {
  return a - b;
};

// 3. Arrow function (strelkali)
const kopaytirish = (a, b) => a * b; // bitta ifoda – return avtomatik
const kvadrat = x => x * x; // bitta parametr – qavs
ixtiyoriy
const yaratish = (ism) => ({ ism, sana: Date.now() }); // obyekt qaytarish –
qavsga o'rang

// Standart parametrlar va rest parametrlar
function salomlash(ism = 'Mehmon', ...qolganlar) {
  return `Salom, ${ism}! Yana ${qolganlar.length} kishi bor.`;
}
salomlash(); // 'Salom, Mehmon! Yana 0 kishi bor.'
salomlash('Ali', 'Vali', 'Gani'); // 'Salom, Ali! Yana 2 kishi bor.'

// Callback – funksiyani argument sifatida berish
function takrorla(n, amal) {
  for (let i = 0; i < n; i++) amal(i);
}
takrorla(3, i => console.log(`${i}-qadam`));
```



Arrow funksiyaning farqi

Arrow funksiyalarda o'zining `this`, `arguments` yo'q — ular tashqi muhitnikini oladi. Shuning uchun obyekt metodlari uchun oddiy funksiya, callback'lar uchun arrow funksiya qulay.

Scope, hoisting va closure

Scope — ko'rinish doirasi

JavaScript

```
const global = 'hamma joyda ko\'rinadi';

function tashqi() {
  const tashqi0zgaruvchi = 'tashqi funksiyada';

  function ichki() {
    const ichki0zgaruvchi = 'faqat ichkida';
    console.log(global, tashqi0zgaruvchi, ichki0zgaruvchi); // ✅ hammasi
    ko\'rinadi
  }
  ichki();
  // console.log(ichki0zgaruvchi); // ❌ ReferenceError
}

if (true) {
  let blokda = 1;
  var funksiyada = 2;
}
// console.log(blokda); // ❌ - let blok bilan cheklangan
console.log(funksiyada); // 2 - var blokni "teshib" chiqadi
```

Ichki doira tashqi doiradagi o'zgaruvchilarni ko'radi, aksincha emas. Bu **leksik scope** deyiladi: o'zgaruvchining ko'rinishi kod *qayerda yozilganiga* bog'liq, qayerda chaqirilganiga emas.

Closure (yopilish)

Closure — funksiya o'zi yaratilgan muhitdagi o'zgaruvchilarni, hatto o'sha muhit tugaganidan keyin ham *eslab qolishi*.

JavaScript

```
function hisoblagichYarat() {  
  let son = 0; // tashqaridan to'g'ridan-to'g'ri o'zgartirib  
  bo'lmaydi (xususiy)  
  return {  
    oshir: () => ++son,  
    kamaytir: () => --son,  
    qiymat: () => son,  
  };  
}  
  
const h1 = hisoblagichYarat();  
h1.oshir(); h1.oshir();  
console.log(h1.qiymat()); // 2  
  
const h2 = hisoblagichYarat(); // yangi, mustaqil "son"  
console.log(h2.qiymat()); // 0
```

Closure'lar hamma joyda: hodisa ishlovchilari, `setTimeout`, React hook'lari, middleware'lar. Ularni tushunish — JavaScript'ni tushunish demak.

Massivlar va ularning metodlari

JavaScript

```
const arr = [10, 20, 30];  
arr.push(40); // oxiriga qo'shish → [10,20,30,40]  
arr.pop(); // oxiridan olish  
arr.unshift(5); // boshiga qo'shish  
arr.shift(); // boshidan olish  
arr.includes(20); // true  
arr.indexOf(30); // 2  
arr.slice(0, 2); // [10, 20] — nusxa, asl massiv o'zgarmaydi  
arr.splice(1, 1); // 1-indeksdan 1 ta elementni O'CHIRADI (asl massivni  
o'zgartiradi)  
arr.join(' - '); // '10 - 30'  
[...arr, 50]; // spread bilan yangi massiv  
Array.from({ length: 5 }, (_, i) => i * 2); // [0, 2, 4, 6, 8]
```

Yuqori tartibli (higher-order) metodlar — eng muhimlari

Bu metodlar asl massivni o'zgartirmaydi va real loyihalarda `for` sikllarining o'rnini deyarli to'liq egallaydi.

```

const mahsulotlar = [
  { nomi: 'Noutbuk', narx: 9_500_000, toifa: 'texnika', soni: 3 },
  { nomi: 'Sichqoncha', narx: 150_000, toifa: 'texnika', soni: 0 },
  { nomi: 'Kitob', narx: 85_000, toifa: 'kitob', soni: 12 },
  { nomi: 'Monitor', narx: 2_800_000, toifa: 'texnika', soni: 5 },
];

// map – har bir elementni o'zgartirib YANGI massiv qaytaradi (uzunlik bir xil)
const nomlar = mahsulotlar.map(m ⇒ m.nomi);
// ['Noutbuk', 'Sichqoncha', 'Kitob', 'Monitor']

// filter – shartga mos elementlar
const mavjud = mahsulotlar.filter(m ⇒ m.soni > 0);

// find / findIndex – birinchi mos element (yoki undefined / -1)
const kitob = mahsulotlar.find(m ⇒ m.toifa === 'kitob');

// some / every – kamida bittasi / hammasi shartga mosmi
mahsulotlar.some(m ⇒ m.soni === 0); // true
mahsulotlar.every(m ⇒ m.narx > 0); // true

// reduce – massivni BITTA qiymatga "yig'ish"
const omborQiymati = mahsulotlar.reduce((jami, m) ⇒ jami + m.narx * m.soni,
0);
// 9_500_000*3 + 0 + 85_000*12 + 2_800_000*5 = 43_520_000

// Toifalar bo'yicha guruhlash (reduce bilan)
const guruhlar = mahsulotlar.reduce((acc, m) ⇒ {
  (acc[m.toifa] ??= []).push(m.nomi);
  return acc;
}, {});
// { texnika: ['Noutbuk', 'Sichqoncha', 'Monitor'], kitob: ['Kitob'] }

// sort – DIQQAT: asl massivni o'zgartiradi. toSorted() – nusxa qaytaradi
const arzondan = [...mahsulotlar].sort((a, b) ⇒ a.narx - b.narx);
const alifbo = mahsulotlar.toSorted((a, b) ⇒ a.nomi.localeCompare(b.nomi));

// Zanjir (chaining)
const texnikaNomi = mahsulotlar
  .filter(m ⇒ m.toifa === 'texnika' && m.soni > 0)

```

```
.sort((a, b) => b.narx - a.narx)
.map(m => `${m.nomi} - ${m.narx.toLocaleString()} so'm`);
```

Obyektlar

JavaScript

```
const talaba = {
  ism: 'Nodira',
  yosh: 21,
  'sevimli fan': 'Matematika', // bo'sh joyli kalit - qo'shtirnoqda
  manzil: { shahar: 'Buxoro', kocha: 'Mustaqillik 5' },
  tanishtir() { // metod
    return `Men ${this.izm}, ${this.yosh} yoshdaman`;
  },
};

talaba.izm; // nuqta orqali
talaba['sevimli fan']; // qavs orqali
const kalit = 'yosh';
talaba[kalit]; // dinamik kalit
talaba.kurs = 3; // yangi xususiyat qo'shish
delete talaba.kurs; // o'chirish
'izm' in talaba; // true

Object.keys(talaba); // ['izm', 'yosh', 'sevimli fan', 'manzil', 'tanishtir']
Object.values({ a: 1, b: 2 }); // [1, 2]
Object.entries({ a: 1, b: 2 }); // [['a', 1], ['b', 2]]
Object.fromEntries(['a', 1]); // { a: 1 }

// Qisqa yozuv
const ism = 'Jamshid', yosh = 30;
const odam = { ism, yosh }; // { ism: 'Jamshid', yosh: 30 }

// Spread bilan birlashtirish / yangilash (immutable usul)
const yangilangan = { ...talaba, yosh: 22, manzil: { ...talaba.manzil, shahar: 'Toshkent' } };
```

Destructuring

JavaScript

```
const { ism, yosh, manzil: { shahar }, kurs = 1 } = talaba; // standart
qiymat bilan

const { ism: talabaIsmi } = talaba; // boshqa
nomga

const [birinchi, ikkinchi, ...qolgan] = [10, 20, 30, 40];
let p = 1, q = 2;
[p, q] = [q, p]; // qiymatlarni almashtirish

// Funksiya parametrlarida — juda ko'p ishlatiladi (ayniqsa React'da)
function kartaChiz({ nomi, narx, chegirma = 0 }) {
  return `${nomi}: ${narx * (1 - chegirma)} so'm`;
}
kartaChiz({ nomi: 'Kitob', narx: 50000, chegirma: 0.1 });
```

JSON

JSON — obyektlarni matn ko'rinishida uzatish formati. Frontend va backend aynan JSON orqali "gaplashadi".

JavaScript

```
const obj = { ism: 'Ali', ballar: [90, 85], faol: true };
const matn = JSON.stringify(obj); // '{"ism":"Ali","ballar":
[90,85],"faol":true}'
const chiroyli = JSON.stringify(obj, null, 2); // 2 bo'shliq bilan formatlangan
const qaytaObj = JSON.parse(matn); // yana obyekt
// JSON'da: kalitlar faqat qo'shtirnoqda, funksiya va undefined bo'lmaydi, izoh
yozib bo'lmaydi
```

Sinflar (class) va OOP

JavaScript

```
class Hisob {
  #balans = 0; // # – haqiqiy xususiy (private) maydon
  static bankNomi = 'Ishonch Bank'; // sinfga tegishli, obyektga emas

  constructor(egasi, boshlangich = 0) {
    this.egasi = egasi;
    this.#balans = boshlangich;
    this.tarix = [];
  }

  get balans() { // getter – xususiyat kabi o'qiladi:
    hisob.balans
    return this.#balans;
  }

  toldirish(summa) {
    if (summa ≤ 0) throw new Error('Summa musbat bo\'lishi kerak');
    this.#balans += summa;
    this.tarix.push({ tur: 'kirim', summa, sana: new Date() });
    return this; // zanjir uchun
  }

  yechish(summa) {
    if (summa > this.#balans) throw new Error('Mablag\' yetarli emas');
    this.#balans -= summa;
    this.tarix.push({ tur: 'chiqim', summa, sana: new Date() });
    return this;
  }
}

// Meros (inheritance)
class JamgarmaHisob extends Hisob {
  constructor(egasi, foiz) {
    super(egasi); // ota konstruktorini chaqirish majburiy
    this.foiz = foiz;
  }

  foizQosh() {
    return this.toldirish(this.balans * this.foiz / 100);
  }
}
```

```

}

const h = new JamgarmaHisob('Laylo', 12);
h.toldirish(1_000_000).foizQosh();
console.log(h.balans);           // 1120000
// console.log(h.#balans);      // ❌ SyntaxError – tashqaridan kirib bo'lmaydi
h instanceof Hisob;             // true

```

Xatolar bilan ishlash

JavaScript

```

function yoshniTekshir(yosh) {
  if (typeof yosh !== 'number' || Number.isNaN(yosh)) {
    throw new TypeError('Yosh son bo\'lishi kerak');
  }
  if (yosh < 0 || yosh > 150) {
    throw new RangeError('Yosh 0 va 150 oralig\'ida bo\'lishi kerak');
  }
  return true;
}

try {
  yoshniTekshir(-5);
} catch (err) {
  console.error(err.name, err.message); // RangeError Yosh 0 va 150
  oralig'ida...
} finally {
  console.log('Bu blok har doim bajariladi (resurslarni tozalash uchun)');
}

// O'z xato sinfigiz
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

```

Modullar (ES Modules)

Katta dasturni fayllarga bo'lish uchun modullar ishlatiladi.

utils/format.js

```
// Nomli eksport (bir nechta bo'lishi mumkin)
export const valyuta = (son) => `${son.toLocaleString('uz-UZ')} so'm`;
export function sanaFormat(d) {
  return new Date(d).toLocaleDateString('uz-UZ');
}

// Standart eksport (faylda bitta)
export default function slugify(matn) {
  return matn.toLowerCase().trim().replace(/[a-z0-9]+/g, '-');
}
```

main.js

```
import slugify, { valyuta, sanaFormat as sana } from './utils/format.js';
import * as fmt from './utils/format.js';

console.log(valyuta(150000)); // '150 000 so'm'
console.log(slugify('Salom Dunyo')); // 'salom-dunyo'
```

Brauzerda modullarni ishlatish: `<script type="module" src="main.js"></script>`. Node.js'da esa `package.json` ga `"type": "module"` qo'shiladi (8-dars).

Amaliy misollar

MISOL 1 Internet-do'kon savatchasi hisob-kitobi

Real e-commerce mantiq: chegirmalar, promokod, yetkazib berish narxi va QQS.

```
const savat = [
  { id: 1, nomi: 'Simsiz qu'loqchin', narx: 450_000, soni: 2, chegirma: 0.1 },
  { id: 2, nomi: 'Telefon g\'ilofi', narx: 60_000, soni: 3, chegirma: 0 },
  { id: 3, nomi: 'Quvvatlagich 65W', narx: 280_000, soni: 1, chegirma: 0.15 },
];

const PROMOKODLAR = {
  BAHOR10: { tur: 'foiz', qiymat: 10 },
  MINUS50K: { tur: 'summa', qiymat: 50_000, minSumma: 500_000 },
};

const BEPUL_YETKAZISH_CHEGARASI = 1_000_000;
const YETKAZISH_NARXI = 25_000;

function qatorSummasi({ narx, soni, chegirma = 0 }) {
  return Math.round(narx * (1 - chegirma)) * soni;
}

function promokodQoll(summa, kod) {
  if (!kod) return { summa, xabar: null };
  const promo = PROMOKODLAR[kod.trim().toUpperCase()];
  if (!promo) return { summa, xabar: 'Promokod topilmadi' };
  if (promo.minSumma && summa < promo.minSumma) {
    return { summa, xabar: `Promokod ${promo.minSumma.toLocaleString()} so'mdan yuqori xaridga amal qiladi` };
  }
  const yangi = promo.tur === 'foiz'
    ? Math.round(summa * (1 - promo.qiymat / 100))
    : summa - promo.qiymat;
  return { summa: Math.max(yangi, 0), xabar: 'Promokod qo\'llandi ✅' };
}

function hisobla(savat, promokod) {
  const qatorlar = savat.map(m => ({ ...m, jami: qatorSummasi(m) }));
  const oraliq = qatorlar.reduce((s, q) => s + q.jami, 0);
  const tejaldi = savat.reduce((s, m) => s + m.narx * m.soni, 0) - oraliq;
  const { summa: promodan, xabar } = promokodQoll(oraliq, promokod);
  const yetkazish = promodan >= BEPUL_YETKAZISH_CHEGARASI ? 0 : YETKAZISH_NARXI;
  const jami = promodan + yetkazish;
```

```

const mahsulotSoni = savat.reduce((s, m) => s + m.soni, 0);

return { qatorlar, oraliq, tejalldi, promokodXabari: xabar, yetkazish,
jami, mahsulotSoni };
}

const f = (n) => `${n.toLocaleString('uz-UZ')} so'm`;
const natija = hisobla(savat, 'bahor10');

console.table(natija.qatorlar.map(({ nomi, soni, jami }) => ({ nomi, soni,
jami: f(jami) })));
console.log(`Mahsulotlar: ${natija.mahsulotSoni} ta`);
console.log(`Oraliq summa: ${f(natija.oraliq)} (chegirmalar:
-${f(natija.tejalldi)})`);
console.log(natija.promokodXabari);
console.log(`Yetkazish: ${natija.yetkazish ? f(natija.yetkazish) :
'BEPUL'}`);
console.log(`To'lovga: ${f(natija.jami)}`);

```

Nimani o'rgandik: `map` / `reduce` bilan hisob-kitob, destructuring, standart parametrlar, obyektidan qidiruv-jadval (lookup table) sifatida foydalanish, sof funksiyalar (bir xil kirish → bir xil chiqish), raqamli ajratgich (`450_000`). Bu kod 13-darsda React interfeysiga ulanadi.

grades.js

```
const talabalar = [
  { ism: 'Akmal', baholar: { matematika: 92, fizika: 78, ingliz: 85 } },
  { ism: 'Barno', baholar: { matematika: 67, fizika: 88, ingliz: 95 } },
  { ism: 'Davron', baholar: { matematika: 45, fizika: 52, ingliz: 60 } },
  { ism: 'Gulnoza', baholar: { matematika: 98, fizika: 94, ingliz: 91 } },
  { ism: 'Erkin', baholar: { matematika: 73, fizika: 39, ingliz: 70 } },
];

const ortacha = (sonlar) => sonlar.reduce((a, b) => a + b, 0) /
sonlar.length;
const yaxlitla = (n) => Math.round(n * 10) / 10;

function daraja(ball) {
  if (ball ≥ 86) return "A'lo";
  if (ball ≥ 71) return 'Yaxshi';
  if (ball ≥ 56) return 'Qoniqarli';
  return 'Qoniqarsiz';
}

// 1. Har bir talabaning o'rtacha bahosi va darajasi
const reyting = talabalar
  .map(({ ism, baholar }) => {
    const ort = yaxlitla(ortacha(Object.values(baholar)));
    return { ism, ortacha: ort, daraja: daraja(ort) };
  })
  .sort((a, b) => b.ortacha - a.ortacha)
  .map((t, i) => ({ orin: i + 1, ...t }));

console.table(reyting);

// 2. Fanlar bo'yicha statistika
const fanlar = Object.keys(talabalar[0].baholar);
const fanStatistika = fanlar.map(fan => {
  const ballar = talabalar.map(t => t.baholar[fan]);
  const eng = talabalar.reduce((best, t) => (t.baholar[fan] >
best.baholar[fan] ? t : best));
  return {
    fan,
```

```

    ortacha: yaxlitla(ortacha(ballar)),
    max: Math.max(...ballar),
    min: Math.min(...ballar),
    eng_yaxshi: eng.ism,
    yiqilganlar: talabalar.filter(t => t.baholar[fan] < 56).map(t =>
t.ism).join(', ') || '-',
  };
});
console.table(fanStatistika);

// 3. Darajalar bo'yicha taqsimot: { "A'lo": 1, Yaxshi: 2, ... }
const taqsimot = reyting.reduce((acc, { daraja }) => {
  acc[daraja] = (acc[daraja] || 0) + 1;
  return acc;
}, {});
console.log(taqsimot);

// 4. Stipendiya oladiganlar: barcha fanlardan 71+
const stipendiya = talabalar
  .filter(t => Object.values(t.baholar).every(b => b >= 71))
  .map(t => t.ism);
console.log('Stipendiya:', stipendiya); // ['Akmal', 'Gulnoza']

```

MISOL 3 Closure bilan amaliy vositalar: memoize, once va debounce

Professional kodda tez-tez uchraydigan uchta yordamchi funksiya — barchasi closure asosida.

```
// 1. memoize — og'ir hisob natijasini keshlash
function memoize(fn) {
  const kesh = new Map(); // closure ichida saqlanadi
  return function (...args) {
    const kalit = JSON.stringify(args);
    if (kesh.has(kalit)) return kesh.get(kalit);
    const natija = fn(...args);
    kesh.set(kalit, natija);
    return natija;
  };
}

const fib = memoize(n => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
console.time('fib');
console.log(fib(90)); // 2880067194370816000 — bir zumda (keshsiz yillab
hisoblanardi)
console.timeEnd('fib');

// 2. once — funksiyani faqat bir marta bajarish (masalan "To'lash"
tugmasi)
function once(fn) {
  let bajarildi = false, natija;
  return (...args) => {
    if (!bajarildi) {
      bajarildi = true;
      natija = fn(...args);
    }
    return natija;
  };
}

const tolash = once((summa) => { console.log(`${summa} so'm to'landi`);
return 'OK'; });
tolash(100000); // "100000 so'm to'landi"
tolash(100000); // hech narsa — ikkinchi marta bajarilmaydi

// 3. debounce — ketma-ket chaqiruvlarda faqat oxirgisini bajarish
// (qidiruv maydoniga yozilganda har harf uchun emas, yozib bo'lgach
so'rov yuborish)
function debounce(fn, kutish = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
```

```
    timer = setTimeout(() => fn(...args), kutish);  
  };  
}  
const qidir = debounce((soz) => console.log('Serverga so\'rov:', soz),  
500);  
qidir('j'); qidir('ja'); qidir('jav'); qidir('java');  
// 500ms dan keyin faqat bitta: "Serverga so'rov: java"
```

Uchala funksiya ham ichki holatni (`kesh`, `bajarildi`, `timer`) closure orqali saqlaydi. `debounce` ni 6-darsda real qidiruv bilan ishlatamiz.

library.js

```
class KitobMavjudEmasError extends Error {
  constructor(isbn) {
    super(`ISBN ${isbn} bo'yicha bo'sh nusxa yo'q`);
    this.name = 'KitobMavjudEmasError';
  }
}

class Kitob {
  constructor({ isbn, nomi, muallif, nusxalar = 1 }) {
    Object.assign(this, { isbn, nomi, muallif });
    this.jami = nusxalar;
    this.bosh = nusxalar;
  }
  get mavjudmi() { return this.bosh > 0; }
  toString() { return `${this.nomi} - ${this.muallif} (${this.bosh}/${this.jami})`; }
}

class Kutubxona {
  #kitoblar = new Map(); // isbn → Kitob
  #ijaralar = []; // { isbn, oquvchi, berilgan, muddat }
  static IJARA_KUNI = 14;
  static JARIMA_KUNIGA = 2000;

  qoshish(malumot) {
    const mavjud = this.#kitoblar.get(malumot.isbn);
    if (mavjud) {
      mavjud.jami += malumot.nusxalar ?? 1;
      mavjud.bosh += malumot.nusxalar ?? 1;
    } else {
      this.#kitoblar.set(malumot.isbn, new Kitob(malumot));
    }
    return this;
  }

  qidirish(soz) {
    const q = soz.toLowerCase();
    return [...this.#kitoblar.values()].filter(
```

```

        k ⇒ k.nomi.toLowerCase().includes(q) ||
k.muallif.toLowerCase().includes(q)
    );
}

berish(isbn, oquvchi, sana = new Date()) {
    const kitob = this.#kitoblar.get(isbn);
    if (!kitob) throw new Error(`Bunday kitob yo'q: ${isbn}`);
    if (!kitob.mavjudmi) throw new KitobMavjudEmasError(isbn);
    kitob.bosh--;
    const muddat = new Date(sana);
    muddat.setDate(muddat.getDate() + Kutubxona.IJARA_KUNI);
    this.#ijaralar.push({ isbn, oquvchi, berilgan: sana, muddat });
    return muddat;
}

qaytarish(isbn, oquvchi, sana = new Date()) {
    const i = this.#ijaralar.findIndex(r ⇒ r.isbn === isbn && r.oquvchi
    === oquvchi);
    if (i === -1) throw new Error('Bunday ijara topilmadi');
    const [ijara] = this.#ijaralar.splice(i, 1);
    this.#kitoblar.get(isbn).bosh++;
    const kechikish = Math.ceil((sana - ijara.muddat) / 86_400_000);
    return kechikish > 0 ? kechikish * Kutubxona.JARIMA_KUNIGA : 0;
}

get hisobot() {
    const kitoblar = [...this.#kitoblar.values()];
    return {
        nomlar: kitoblar.length,
        jamiNusxa: kitoblar.reduce((s, k) ⇒ s + k.jami, 0),
        ijarada: this.#ijaralar.length,
        tugaganlar: kitoblar.filter(k ⇒ !k.mavjudmi).map(String),
    };
}

// Foydalanish
const lib = new Kutubxona()
    .qoshish({ isbn: '978-1', nomi: "O'tkan kunlar", muallif: 'Abdulla
Qodiriy', nusxalar: 2 })
    .qoshish({ isbn: '978-2', nomi: 'Sariq devni minib', muallif:
"Xudoyberdi To'xtaboyev" })
    .qoshish({ isbn: '978-3', nomi: 'Clean Code', muallif: 'Robert Martin'

```

```
});

lib.berish('978-2', 'Sherzod', new Date('2026-09-01'));
try {
  lib.berish('978-2', 'Madina');
} catch (e) {
  if (e instanceof KitobMavjudEmasError) console.warn('⚠️', e.message);
  else throw e; // kutilmagan xatoni yashirmaymiz
}

const jarima = lib.qaytarish('978-2', 'Sherzod', new Date('2026-09-20'));
console.log(`Jarima: ${jarima} so'm`); // 5 kun kechikish → 10000 so'm
console.log(lib.qidirish('qodiriy').map(String));
console.log(lib.hisobot);
```

Ko'rib chiqilgan tushunchalar: xususiy maydonlar (`#`), statik konstantalar, getter, o'z xato sinfi va `instanceof` bilan tekshirish, `Map`, sanalar bilan hisob, metodlar zanjiri. 10-darsda xuddi shu mantiqni ma'lumotlar bazasiga o'tkazamiz.

Mashqlar



Mustaqil ish

- `sozlarSoni(matn)` funksiyasini yozing: matndagi har bir so'z necha marta uchrashini obyekt ko'rinishida qaytarsin (katta-kichik harf farqsiz, tinish belgilarisiz).
- Massivdagi takroriy elementlarni olib tashlang — ikki usulda: `filter + indexOf` va `new Set()`.
- 1-misolga "bir nechta promokod qo'llab bo'lmaydi" va "chegirmali mahsulotga promokod amal qilmaydi" qoidalarini qo'shing.
- `throttle(fn, ms)` funksiyasini yozing: `fn` har `ms` millisoniyada ko'pi bilan bir marta bajarilsin. U `debounce` dan nimasi bilan farq qiladi?
- `TodoList` sinfini yozing: `add`, `toggle(id)`, `remove(id)`, `filter('all'|'done'|'active')` metodlari va `stats` getteri bilan.

▼ 1-mashq yechimi

JavaScript

```
const sozlarSoni = (matn) =>
  matn.toLowerCase()
    .replace(/[^\p{L}\p{N}\s'-]/gu, '')
    .split(/\s+/)
    .filter(Boolean)
    .reduce((acc, soz) => ({ ...acc, [soz]: (acc[soz] || 0) + 1 }),
  {});

sozlarSoni('Bir, ikki. Bir uch! bir'); // { bir: 3, ikki: 1, uch: 1 }
```

✓ Xulosa

- `const` > `let`; `var` ishlatmang. Har doim `≡`.
- Primitivlar qiymat bo'yicha, obyektlar havola bo'yicha nusxalanadi.
- `??`, `?.`, destructuring va spread — zamonaviy JS'ning kundalik vositalari.
- `map`, `filter`, `reduce`, `find`, `some` / `every` — ma'lumotlar bilan ishlashning asosi.
- Closure — funksiya o'z muhitini eslab qoladi; xususiy holat va yordamchi funksiyalar uchun ishlatiladi.
- Sinflar, `#private` maydonlar, `extends`; xatolarni `throw` / `try...catch` bilan boshqaring.

☐ Darsni o'qib bo'ldim